day 1

Day 1 — SELECT Basics + What is a Database

Part 1: DBMS Theory (15 min)

What is a database? A database is simply an organized collection of data. A spreadsheet is technically a database — but it breaks down when you have millions of rows, multiple users, or need to enforce rules on the data.

What is a DBMS? A Database Management System is the *software* that manages the database. It handles storage, retrieval, security, concurrency (multiple users at once), and data integrity. PostgreSQL is a DBMS. The actual data files sitting on disk are the database — PostgreSQL is the system that manages them.

What is an RDBMS? A *Relational* DBMS organizes data into **tables** (also called relations). Each table has **rows** (records) and **columns** (fields/attributes). Tables can relate to each other — that's where the "relational" part comes from. PostgreSQL, MySQL, SQLite are all RDBMS.

Client-Server Model: PostgreSQL runs as a **server** (a background process listening for connections). You connect to it using a **client** — that's what `psql` is. Your Docker setup runs the server inside a container, and when you do `docker exec -it pg_learn psql ...`, you're launching a client that connects to it.

```
You (typing SQL) → psql (client) → PostgreSQL (server) → Data files on disk
```

What is SQL? Structured Query Language — the language you use to talk to the RDBMS. You write SQL, the client sends it to the server, the server processes it and returns results.

Part 2: Let's Set Up a Practice Table (10 min)

Run this in your `psql` terminal to create a table we'll practice with today:

```sql
CREATE TABLE employees (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50),
    salary INTEGER,
    city VARCHAR(50),
    hire_date DATE
);

INSERT INTO employees (name, department, salary, city, hire_date) VALUES
('Arun Kumar',     'Engineering', 95000,  'Bangalore', '2021-03-15'),
('Priya Sharma',   'Engineering', 88000,  'Mumbai',    '2022-01-10'),
('Rahul Verma',    'Marketing',   72000,  'Delhi',     '2020-07-22'),
('Sneha Patel',    'Marketing',   68000,  'Bangalore', '2023-02-01'),
('Vikram Singh',   'Sales',       75000,  'Mumbai',    '2019-11-30'),
('Anita Desai',    'Sales',       71000,  'Delhi',     '2021-08-14'),
('Karan Mehta',    'Engineering', 102000, 'Bangalore', '2018-05-20'),
('Deepa Nair',     'HR',          65000,  'Mumbai',    '2022-09-05'),
('Amit Joshi',     'HR',          62000,  'Delhi',     '2023-06-12'),
('Neha Gupta',     'Engineering', 91000,  'Mumbai',    '2020-04-18');
```

Verify it worked:

```
SELECT * FROM employees;
```

You should see 10 rows. If yes, you're ready.

Part 3: SELECT Fundamentals (30 min)

3.1 — SELECT and FROM

`SELECT` chooses *which columns* to show. `FROM` says *which table*.

```
-- Get all columns
SELECT * FROM employees;

-- Get specific columns only
SELECT name, salary FROM employees;
```

Think of `SELECT` as "show me these columns" and `FROM` as "from this table."

Rule of thumb: Avoid `SELECT *` in real projects — always name the columns you need. It's faster and clearer.

3.2 — WHERE (filtering rows)

`WHERE` filters which rows come back. Only rows that match the condition are returned.

```
-- Only employees in Engineering
SELECT name, department, salary
FROM employees
WHERE department = 'Engineering';
```

Result:

```
    name        | department   | salary
---------------+-------------+--------
 Arun Kumar     | Engineering | 95000
 Priya Sharma   | Engineering | 88000
 Karan Mehta    | Engineering | 102000
 Neha Gupta     | Engineering | 91000
```

```
-- Employees earning more than 80000
SELECT name, salary
FROM employees
WHERE salary > 80000;
```

Common comparison operators: `=`, `!=` (or `<>`), `>`, `<`, `>=`, `<=`

3.3 — Combining conditions: AND, OR, NOT

```
-- Engineering AND salary above 90000
SELECT name, salary
FROM employees
WHERE department = 'Engineering' AND salary > 90000;
```

Returns: Arun Kumar (95000), Karan Mehta (102000), Neha Gupta (91000)

```sql
-- In Mumbai OR in Delhi
SELECT name, city
FROM employees
WHERE city = 'Mumbai' OR city = 'Delhi';
```

```sql
-- Everyone NOT in Sales
SELECT name, department
FROM employees
WHERE NOT department = 'Sales';
```

Operator precedence: `AND` binds tighter than `OR`. Use parentheses to be safe:

```sql
-- This might surprise you without parentheses
WHERE department = 'Engineering' OR department = 'HR' AND salary > 70000
-- ↑ This means: Engineering (all) OR (HR with salary > 70000)

-- What you probably meant:
WHERE (department = 'Engineering' OR department = 'HR') AND salary > 70000
```

3.4 — Column Aliases (AS)

Aliases rename columns in the output. They don't change the actual table.

```sql
SELECT name, salary AS monthly_pay, department AS dept
FROM employees;
```

The `AS` keyword is optional — `salary monthly_pay` works too — but always use `AS` for clarity.

3.5 — LIMIT

Controls how many rows are returned. Useful for previewing large tables.

```sql
-- Just show the first 3 rows
SELECT name, salary
FROM employees
LIMIT 3;
```

Note: Without `ORDER BY`, the order isn't guaranteed. You'll usually pair `LIMIT` with `ORDER BY` (tomorrow's topic).

3.6 — Putting it together

```sql
SELECT name AS employee, salary, city
FROM employees
WHERE department = 'Engineering' AND city = 'Mumbai'
LIMIT 5;
```

The mental model for reading any query:

1. `FROM` — which table?
2. `WHERE` — which rows survive the filter?
3. `SELECT` — which columns to show?
4. `LIMIT` — how many rows max?

This is actually close to the order PostgreSQL processes the query internally too.

Part 4: Exercises (45 min)

Try these in your psql terminal. Write the query yourself before checking the answer.

Q1. Show the name and city of all employees who work in Marketing.

Q2. Show the name and salary of all employees in Bangalore who earn more than 90,000.

Q3. Show the name, department, and salary of employees who are either in HR or earn more than 95,000. Alias the name column as `employee_name`.

Q4. Show all columns for employees who are NOT in Bangalore and NOT in Mumbai. (Hint: there are multiple ways to write this)

Q5. Show the name and hire_date of any 3 employees from Engineering.

When you're done (or stuck), share your answers and I'll review them. Or say **"show answers"** and I'll walk through each solution.